

Lab: Digital Signatures Using OpenSSL

Estimated Time: 1 hour

Overview

This lab demonstrates how to create and verify digital signatures using OpenSSL. Digital signatures ensure data integrity, authenticity, and non-repudiation. You will generate a key pair, sign a file, and verify the signature step-by-step.

Learning objectives

By the end of this lab, you will be able to:

- Generate and use key pairs for digital signatures
- Sign files and verify their authenticity using OpenSSL
- Understand the role of hashing in digital signatures

Prerequisites (Optional)

- Install OpenSSL on your system.
 - Basic understanding of cryptographic keys and digital signatures is advantageous.
-

Initializing the Lab Environment

Follow the below steps to initialize the lab environment.

- Select **Terminal** and then select **New Terminal**. This will open a new terminal.

FileEditSelectionViewGoRunTerminalHelp

←→

☐

SKILLS ...

> DATABASES

> BIG DATA

> CLOUD

> EMBEDDABLE AI

> OTHER

Launch Appli...

Welcome X

Cloud IDE

Version 1.7.4

VS Code API Versio

What is Cloud

Cloud IDE is a full

Environment (IDE

debug your code

Getting Started

On your left,you'll find the instructions for the tasks you need to complete in this lab. Follow them step-by-step to achieve the objectives of this lab. You can also click the chat button to ask for help from Tai, your AI Teaching Assistant at any time.

On your right (here),is the Cloud IDE tool, a fully-featured Integrated Development Environment (IDE) that you'll be using to write, run, and debug your code.

Here are some key components of the Cloud IDE tool:

- **File Explorer:** Navigate through your files and directories. [Click HERE to open the File Explorer.](#)
- **Terminal:** Run your code and see the output here. [Click HERE to open a New Terminal.](#)
- **Skills Network Toolbox:** Access various technologies like databases and Embeddable AI with just a click. [Click HERE to toggle the Skills Network Toolbox.](#)

☒ Show welcome page on startup

New TerminalCtrl+Shift+`

New Terminal (With Profile)...

Choose Default Profile...

Run Task...

Run Build Task

Run Test Task

Rerun Last TaskCtrl+Shift+K

Show Running Tasks...

Restart Running Task...

Terminate Task...

Attach Task...

Configure Tasks...



SkillsNetwork

Open Workspace

Recent

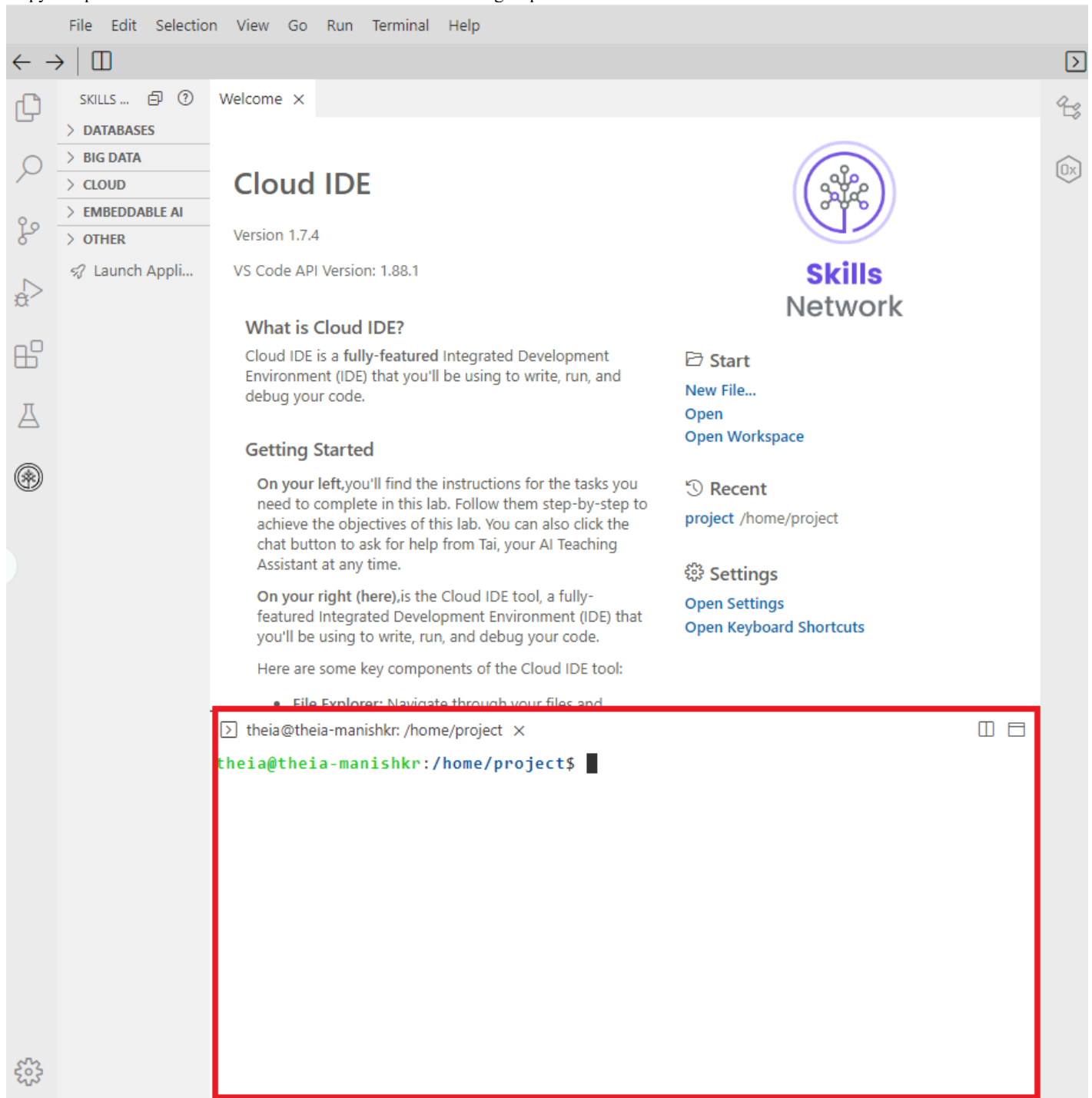
project /home/project

Settings

Open Settings

Open Keyboard Shortcuts

- Copy and paste the commands into the terminal for the remaining steps.



Step 1: Generate a Private Key

Run the below command to create a private key to be used for signing.

```
openssl genpkey -algorithm RSA -out private_key.pem -pkeyopt rsa_keygen_bits:2048
```

Command breakdown:

- **genpkey**: Generates a private key.
 - **-algorithm RSA**: Specifies RSA as the key generation algorithm.
 - **-out private_key.pem**: Saves the private key to the file `private_key.pem`.
 - **-pkeyopt rsa_keygen_bits:2048**: Generates a 2048-bit RSA key for strong encryption.
-

Step 2: Extract the Public Key

Run the below command to extract the public key from the private key:

```
openssl rsa -in private_key.pem -pubout -out public_key.pem
```

Command breakdown:

- **rsa -in private_key.pem**: Reads the private key file.
 - **-pubout**: Outputs the public key.
 - **-out public_key.pem**: Saves the public key to the file `public_key.pem`.
-

Step 3: Create a Test File

Run the below command to create a file for signing in:

```
echo "This is a file to demonstrate digital signatures." > testfile.txt
```

Command breakdown:

- **echo**: Outputs the text.
 - **>**: Saves the output into a file named `testfile.txt`.
-

Step 4: Generate a Digital Signature

Run the below command to sign the file using the private key:

```
openssl dgst -sha256 -sign private_key.pem -out signature.bin testfile.txt
```

Command breakdown:

- **dgst**: Executes the digest function.
 - **-sha256**: Specifies the hashing algorithm (SHA-256) for the signature.
 - **-sign private_key.pem**: Signs the data using the specified private key.
 - **-out signature.bin**: Saves the generated signature to the file signature.bin.
 - **testfile.txt**: Input file to sign.
-

Step 5: Verify the Digital Signature

Run the below command to use the public key for signature verification:

```
openssl dgst -sha256 -verify public_key.pem -signature signature.bin testfile.txt
```

Command breakdown:

- **-verify public_key.pem**: Uses the public key for signature verification.
- **-signature signature.bin**: Specifies the file containing the signature.
- **testfile.txt**: Verifies against the original file.

Expected Output:

Verified OK (if the signature matches the file).

Step 6: Test Signature Tampering

Follow the below steps for testing signature tampering.

Step A: Modify the file

Append a line to the file:

```
echo "Tampering with the file." >> testfile.txt
```

Step B: Reverify the signature

```
openssl dgst -sha256 -verify public_key.pem -signature signature.bin testfile.txt
```

Expected output:

Verification Failure (since the file has been altered).

Step 7: Examine the Signature

To view the signature in Base64 format:

```
openssl enc -base64 -in signature.bin -out signature.b64
cat signature.b64
```

Command breakdown:

- **enc -base64**: Converts the binary signature to Base64 for readability.
 - **-in and -out**: Specifies input and output files.
 - **cat signature.b64**: Displays the Base64-encoded signature.
-

Exercises

Here are **three unique exercises for the Digital Signatures Lab**, along with hints to foster deeper understanding:

Exercise 1: Verify a Digital Signature without Access to the Private Key

Objective: Verify the authenticity of a message using only the sender's public key and the provided digital signature.

Steps hint:

- Start with a signed message and a public key file.
- Use OpenSSL to verify the signature against the original message.

Hints:

1. Use the `openssl dgst` command with the `-verify` option to check signature.
2. Ensure the original message, signature, and public key are all available for the verification.
3. Use the same hashing algorithm as the one used to generate the signature.

Example commands:

- Verify the signature:

```
openssl dgst -sha256 -verify public.key -signature message.sig message.txt
```

Exercise 2: Experiment with Tampered Data Verification

Objective: Understand the integrity property of digital signatures by modifying the original message and observing the verification process.

Steps hint:

- Generate a digital signature for a message.
- Alter the message slightly, for example, change a word or a character).
- Attempt to verify the signature using the altered message and note the result.

Hints:

1. Use a text editor or sed command to modify the message.
2. Perform verification as shown in Exercise 1 with the tampered message.
3. Observe how even a minor change invalidates the signature.

Example commands:

- Alter the message:

```
sed 's/word1/word2/' message.txt > tampered_message.txt
```

- Verify with tampered data:

```
openssl dgst -sha256 -verify public.key -signature message.sig tampered_message.txt
```

Exercise 3: Simulate a Real-World Workflow for Digital Signatures

Objective: Create a workflow where one user signs a file, and another user verifies it, mimicking a real-world document signing process.

Steps Hint:

- User 1 generates a key pair and signs a document.
- User 2 receives the document, signature, and public key for performing verification.

Hints:

1. Simulate User 1 generating the signature:

```
openssl dgst -sha256 -sign private.key -out document.sig document.txt
```

2. Simulate User 2 verifying the signature:

```
openssl dgst -sha256 -verify public.key -signature document.sig document.txt
```

3. Exchange files (for example, through file-sharing tools) to replicate real-world scenarios.

Conclusion

This lab introduced the process of creating and verifying digital signatures using OpenSSL. By following the steps given in this lab, you learned how digital signatures ensure data integrity and authenticity, a critical component of modern cryptography.

Note: This version of the lab provides detailed explanations for each command and additional exercises for better understanding.

Author(s)

Dr. Manish Kumar

© IBM Corporation. All rights reserved.